

پروژه‌ی درس معماری کامپیوتر – دکتر جهانگیر

کالبدشکافی پردازنده: از اندازه‌گیری تا بهبود

۱. شرح کلی

در این پروژه، شما نقش یک کارآگاه سخت‌افزار را بر عهده می‌گیرید. پردازنده‌ای که هر روز با آن کار می‌کنید، در ظاهر یک جعبه‌ی سیاه است؛ می‌دانید چه کاری انجام می‌دهد، اما از آنچه در ریزمعماری درونش می‌گذرد بی‌خبرید. قرار است این جعبه‌ی سیاه را باز کنید. نخست با زمان‌سنجی و طراحی آزمایش‌های دقیق، رفتار پنهان پردازنده را آشکار می‌کنید؛ سپس با کمک شمارنده‌های سخت‌افزاری درون پردازنده (Hardware Performance Counters) همان رفتار را این بار مستقیم و با عدد مشاهده می‌کنید؛ و در پایان، از این شناخت برای سریع‌تر کردن برنامه‌ی خودتان بهره می‌گیرید.

پروژه از سه بخش اصلی به‌هم‌پیوسته و یک بخش امتیازی تشکیل شده است. در بخش اول، سلسله‌مراتب حافظه‌ی ماشین خود را تنها با زمان‌سنجی استخراج می‌کنید. از اندازه‌ی سطوح مختلف cache و اندازه‌ی خط حافظه گرفته تا درجه‌ی associativity و اثر TLB. در بخش دوم، سراغ شمارنده‌های سخت‌افزاری می‌روید و با ابزارهایی مانند perf در لینوکس یا Intel VTune معیارهای واقعی ریزمعماری را اندازه می‌گیرید. نرخ برخورد و فقدان در هر سطح cache، تعداد پیش‌بینی‌های درست و نادرست پرش، و تعداد دستور اجراشده به ازای هر کلاک (IPC). در بخش سوم، یک پدیده‌ی عملکردی غیرشهودی را برمی‌گزینید، علت آن را نه با حدس بلکه با استناد به همین شمارنده‌ها اثبات می‌کنید و سپس برنامه را بهبود می‌دهید. سرانجام، بخش امتیازی به موضوع کنترل هسته‌ها می‌پردازد.

۲. بخش اول: استخراج اثرانگشت ریزمعماری با زمان‌سنجی

در این بخش برنامه‌ای می‌نویسید که روی پردازنده‌ی شخصی خودتان اجرا شود و تنها با اندازه‌گیری زمان دسترسی به حافظه، پارامترهای پنهان سیستم را بیرون بکشد؛ بی‌آنکه به هیچ مشخصات یا ابزار بیرونی تکیه کند.

ایده‌ی محوری

آرایه‌هایی با اندازه‌های مختلف را به‌صورت مکرر پیمایش کرده و میانگین زمان هر دسترسی را اندازه می‌گیرید. تا زمانی که آرایه به‌طور کامل در L1 cache جای می‌گیرد، دسترسی‌ها بسیار سریع‌اند. به محض بزرگ‌تر شدن آرایه از ظرفیت L1، بخشی از دسترسی‌ها به L2 منتقل می‌شوند و میانگین زمان افزایش می‌یابد. این جهش‌ها، اندازه‌ی هر سطح را مشخص می‌کنند.

موارد قابل استخراج

- اندازه‌ی سطوح cache (L1, L2, L3): از روی محل جهش‌ها در نمودار زمان دسترسی برحسب اندازه‌ی working set.
- اندازه‌ی خط حافظه (cache line size): با تغییر گام پیمایش و یافتن گامی که از آن پس هر دسترسی یک miss است.

- درجه‌ی associativity و اثر TLB: با طراحی آزمایشی که در آن الگوی دسترسی عمداً چند خانه را به یک set می‌نگارد، می‌توانید درجه‌ی associativity را برآورد کنید و مرز اثرگذاری TLB را نیز مشخص نمایید.

راهنمایی

- پیاده‌سازی بر عهده‌ی خودتان است و همین، بخش اصلی یادگیری در این پروژه به شمار می‌آید. برای شروع می‌توانید چارچوب زیر را دنبال کنید:
- به‌جای یک حلقه‌ی ساده، یک الگوی دسترسی وابسته بسازید. در پیمایش ساده، پردازنده دسترسی‌ها را با هم و به‌موازات انجام می‌دهد و زمان واقعی هر دسترسی پنهان می‌ماند. ترفند رایج این است که محتوای هر خانه را نشانی خانه‌ی بعدی قرار دهید (pointer chasing)؛ این‌گونه پردازنده مجبور می‌شود پیش از هر دسترسی منتظر نتیجه‌ی دسترسی قبلی بماند و زمان هر گام آشکار می‌شود.
- اندازه‌ی آرایه را روی مقیاس لگاریتمی افزایش دهید. از چند کیلوبایت آغاز کنید و تا چند ده مگابایت پیش بروید و برای هر اندازه، میانگین زمان هر دسترسی را ثبت نمایید.
- پیش از هر اندازه‌گیری، آرایه را چند بار پیمایش کنید تا داده‌ها در cache و TLB بنشینند (warm-up)؛ سپس زمان را ثبت کنید تا خطای مربوط به بارگذاری اولیه وارد نتایج نشود.
- در پایان، اندازه‌های به‌دست‌آمده را با مشخصات رسمی پردازنده مقایسه کنید و هر اختلاف را توضیح دهید؛ همین تحلیل اختلاف، بخش ارزشمندی از کار شماست.
- در انتخاب زبان آزادید، اما زبان‌هایی مانند C، C++ یا Rust به‌دلیل کنترل دقیق‌تر روی زمان و حافظه، برای این نوع زمان‌سنجی مناسب‌ترند.

نکته‌ی مهم

نتایج بر روی هر ماشین اندکی متفاوت‌اند و گاهی جهش‌ها به‌وضوح ظاهر نمی‌شوند (به‌ویژه برای L1 یا روی پردازنده‌های کم‌مصرف). این مورد نقص نیست، بلکه بخشی از واقعیت اندازه‌گیری است. اگر یک جهش مبهم بود، تحلیل علت آن مانند دخالت prefetcher یا تغییر فرکانس پردازنده (frequency scaling) خود بخشی ارزشمند از کار شماست.

۳. بخش دوم: خواندن شمارنده‌های سخت‌افزاری پردازنده

در بخش اول تنها با زمان‌سنجی به رفتار پردازنده پی بردید. در این بخش، مستقیماً به شمارنده‌های سخت‌افزاری درون پردازنده دسترسی پیدا می‌کنید. این شمارنده‌ها رویدادهای واقعی ریزمعماری را می‌شمارند و به شما اجازه می‌دهند به‌جای استنتاج غیرمستقیم، عدد دقیق آنچه درون پردازنده رخ می‌دهد را ببینید.

ابزارها

- در انتخاب ابزار دستتان باز است؛ هر ابزاری که بتواند شمارنده‌های سخت‌افزاری را بخواند برای این کار مناسب است. چند گزینه‌ی رایج عبارت‌اند از:
- perf در لینوکس ساده‌ترین و در دسترس‌ترین گزینه است. با دو دستور perf stat و perf record می‌توان بیشتر معیارهای موردنیاز را گرفت؛ روی اغلب پردازنده‌ها کار می‌کند و به نصب نرم‌افزار خاصی نیاز ندارد.

- Intel VTune Profiler ابزار حرفه‌ای و بصری اینتل برای تحلیل عمیق ریزمعماری است و روی پردازنده‌های اینتل به‌خوبی کار می‌کند. استفاده از آن پیشنهاد می‌شود اما اجباری نیست و perf یا هر ابزار معادلی نیز پذیرفته است.
- روی سایر سکوها هم گزینه هست: در ویندوز می‌توان از Windows Performance Counters یا ابزارهای مشابه، و روی پردازنده‌های AMD از AMD uProf بهره گرفت.

معیارهایی که باید استخراج کنید

- برنامه‌ی بخش اول (یا هر برنامه‌ی نمونه‌ی مناسب دیگری) را زیر ابزار پروفایل اجرا کنید و دست‌کم معیارهای زیر را گزارش دهید:
- نرخ برخورد و فقدان cache در هر سطح: تعداد دسترسی‌ها، تعداد miss و نسبت میان آن‌ها را برای L1، برای LLC و در صورت امکان برای سطوح میانی گزارش کنید.
- پیش‌بینی پرش: تعداد کل پرش‌ها، تعداد پیش‌بینی‌های نادرست (branch mispredictions) و نرخ خطای پیش‌بینی را به‌دست آورید.
- IPC یا CPI: تعداد دستور اجرا شده به ازای هر کلاک، به‌عنوان یک معیار کلی از میزان کارایی اجرا.
- و در صورت تمایل، معیارهای بیشتر: TLB miss، چرخه‌های توقف ناشی از انتظار حافظه (stall cycles) و هر شمارنده‌ی دیگری که به تحلیل شما کمک کند.

پیوند با بخش اول

یافته‌های این بخش را کنار نتایج زمان‌سنجی بخش اول بگذارید و ببینید آیا با هم می‌خوانند: آیا همان‌جا که در نمودار زمان دسترسی جهش دیده‌اید، نرخ فقدان cache نیز بالا رفته است؟ اگر این دو روی هم بیفتند، درستی کار بخش اول را این بار با شهادتی مستقل و مستقیم از خود سخت‌افزار تأیید کرده‌اید.

۴. بخش سوم: تحلیل و بهبود یک پدیده‌ی عملکردی

در این بخش، یک پدیده‌ی عملکردی غیرشهودی را انتخاب می‌کنید؛ موردی که در آن تغییری ظاهراً کم‌اهمیت در کد، عملکرد را به‌طرز چشمگیر (گاه تا چند برابر) تغییر می‌دهد. وظیفه‌ی شما این است که نخست علت آن را بر پایه‌ی شواهد اثبات کنید و سپس با تکیه بر همین شناخت، برنامه را بهبود دهید. استفاده از شمارنده‌های سخت‌افزاری بخش دوم در این بخش الزامی است: فرضیه‌ی خود را باید با عدد شمارنده‌ها (نه فقط زمان اجرا) پشتیبانی کنید.

روش علمی و بهبود موردانتظار

- مشاهده: نخست پدیده را بازتولید کنید و با شواهد کمی (زمان اجرا، عدد و نمودار) نشان دهید که واقعا وجود دارد.
- فرضیه: توضیحی معماری برای علت پدیده پیشنهاد کنید؛ یعنی بگویید کدام رفتار درون پردازنده این تفاوت را می‌سازد.
- آزمایش کنترل‌شده: آزمایشی طراحی کنید که اگر فرضیه‌تان درست باشد نتیجه‌ای مشخص بدهد و اگر نادرست باشد نتیجه‌ای متفاوت؛ سپس نتیجه را با شمارنده‌های سخت‌افزاری بسنجید، نه فقط با زمان اجرا.

- رد فرضیه‌های رقیب: دست کم یک توضیح جایگزین را هم در نظر بگیرید و با شواهد نشان دهید که آن توضیح درست نیست. این مرحله، مهم‌ترین و ارزشمندترین بخش کار شماست.
- نتیجه‌گیری: در پایان، علت را بر پایه‌ی شواهد عددی کامل و دقیق اعلام کنید.
- بهبود: حالا که علت را اثبات کرده‌اید، از همین شناخت برای نوشتن نسخه‌ی بهتری از برنامه استفاده کنید؛ مثلاً با اصلاح الگوی دسترسی به حافظه، کاستن از پرش‌های بدپیش‌بینی، یا بهبود locality. میزان بهبود را هم با زمان اجرا و هم با تغییر شمارنده‌ها (مثلاً کاهش نرخ miss یا کاهش branch misprediction) نشان دهید.

فهرست پدیده‌های پیشنهادی

می‌توانید یکی از پدیده‌های زیر را برگزینید یا اگر ترجیح می‌دهید، پدیده‌ی تازه‌ای از خودتان معرفی و تحلیل کنید:

پدیده	شرح	زمینه‌ی معماری
دسترسی با گام توانی از ۲	افت شدید سرعت هنگام پیمایش آرایه با گامی برابر توانی از ۲.	هم‌نشینی آدرس‌ها در یک set و وقوع conflict miss.
ترتیب حلقه‌ها در ضرب ماتریس	تغییر چشمگیر عملکرد تنها با جابه‌جایی ترتیب حلقه‌های تودرتو.	تفاوت در locality و الگوی دسترسی به حافظه.
پیمایش سطری در برابر ستونی	تفاوت محسوس سرعت میان دسترسی سطری و ستونی به یک ماتریس.	چینش row-major و هدررفتن خط حافظه در دسترسی ستونی.
false sharing	افت کارایی هنگام کار هم‌زمان چند ریشه روی متغیرهای نزدیک به هم.	قرارگرفتن متغیرها روی یک خط حافظه و رقابت هسته‌ها.
پیش‌بینی پرش	تفاوت عملکرد پردازش آرایه‌ی مرتب در برابر نامرتب.	دقت branch predictor روی داده‌ی منظم در برابر تصادفی.

پیوند میان بخش‌ها

اگر پدیده‌ای را انتخاب کنید که به سلسله‌مراتب حافظه مربوط است (مانند پیمایش سطری در برابر ستونی)، نقطه‌ای را که رفتار برنامه در آن تغییر می‌کند به اندازه‌های cache که در بخش اول استخراج کرده‌اید پیوند بزنید و نشان دهید که این نقطه دقیقاً همان جایی است که working set از ظرفیت آن سطح فراتر می‌رود. این پیوند میان بخش‌ها بخش مهمی از تحلیل شماست و ارزش علمی کار را به‌روشنی بالا می‌برد.

۵. بخش امتیازی (نیم‌نمره): کنترل هسته‌ها و ریسمان‌ها

این بخش، اختیاری و دارای نیم نمره‌ی امتیازی است و در واقع سنگین‌ترین و ریزمعماری‌ترین بخش پروژه به شمار می‌آید. هدف این است که این بار به‌جای مشاهده، خودتان دست به کنترل ببرید: تعیین کنید برنامه دقیقاً روی کدام هسته‌های فیزیکی اجرا شود و اثر این تصمیم را بر کارایی بسنجید.

سنجاق کردن ریسمان‌ها به هسته (Thread/Core Affinity)

یک برنامه‌ی چندریسمانی بنویسید و ریسمان‌ها را به هسته‌های مشخصی سنجاق کنید (در لینوکس با `taskset` یا `sched_setaffinity` و در ویندوز با `SetThreadAffinityMask`). آنگاه سه سناریو را با هم مقایسه کنید: نخست، دو ریسمان روی دو هسته‌ی فیزیکی جدا؛ دوم، دو ریسمان روی دو ریسمان منطقی یک هسته‌ی واحد (SMT یا Hyper-Threading)؛ و سوم، رهاکردن کار به دست زمان‌بند سیستم‌عامل. تفاوت کارایی را اندازه بگیرید و با کمک شمارنده‌های بخش دوم و مفهوم اشتراک `cache` و منابع اجرایی، توضیح دهید چرا چنین تفاوتی رخ می‌دهد.

یک پیشنهاد خوب این است که همین‌جا پدیده‌ی `false sharing` یا رقابت بر سر `cache` مشترک را بازتولید کنید و نشان دهید که صرف تغییر چیدمان ریسمان‌ها روی هسته‌ها، چگونه نتیجه را زیر و رو می‌کند.

غیرفعال‌سازی هسته و تحلیل مقیاس‌پذیری

با غیرفعال کردن یک یا چند هسته (در لینوکس از راه `sysfs` و مسیر `/sys/devices/system/cpu/cpuN/online`، یا از طریق تنظیمات سیستم‌عامل و بایوس) رفتار برنامه را به‌ازای تعداد هسته‌های فعال گوناگون بسنجید. سپس نمودار `speedup` برحسب تعداد هسته را رسم کنید، آن را با پیش‌بینی قانون امدال (Amdahl) بسنجید و از روی آن، سهم سریال برنامه‌ی خود را تخمین بزنید.

تحویلی این بخش

تحویلی این بخش شامل روش سنجاق کردن ریسمان‌ها، جدول یا نمودار مقایسه‌ی سناریوها، نمودار `speedup` و تحلیلی است که به شمارنده‌ها و قانون امدال تکیه کند.

۶. الزامات اصالت کار

داده‌های این پروژه باید از پردازنده‌ی شخصی شما به‌دست آیند و قابل بازتولید باشند؛ ارائه‌ی مشخصات دقیق پردازنده برای راستی‌آزمایی الزامی است. ارزیابی بر کیفیت تحلیل، طراحی آزمایش و تفسیر نتایج استوار است، نه صرفاً بر نتیجه‌ی نهایی. در جلسه‌ی تحویل، باید نمودارها و استدلال‌های خود را توضیح دهید و به پرسش‌های مربوط پاسخ گوید. استفاده از هر منبع کمکی، از جمله ابزارهای هوش مصنوعی، را باید در گزارش ذکر کنید.

۷. تحویلی‌ها

- گزارش تحلیلی (۶ تا ۱۰ صفحه) شامل روش کار، نمودارها، خروجی شمارنده‌های سخت‌افزاری، فرضیه‌ها، آزمایش‌ها، بهبود انجام‌شده و نتیجه‌گیری.
- کدهای آزمایش، تمیز و قابل اجرا، به‌همراه راهنمای اجرا.
- نسخه‌ی اولیه و نسخه‌ی بهبودیافته‌ی برنامه‌ی بخش سوم، به‌همراه گزارش میزان بهبود (زمان اجرا و شمارنده‌ها).
- داده‌ی خام، خروجی ابزار پروفایل (perf/VTune) و نمودارهای همه‌ی بخش‌ها.
- مشخصات دقیق پردازنده‌ی مورد استفاده (مدل و مشخصات سطوح `cache`)

۸. شیوهی ارزیابی

وزن	معیار
۲۰٪	صحت استخراج پارامترهای حافظه با زمان سنجی (بخش اول)
۲۵٪	صحت و کامل بودن استخراج شمارنده‌های سخت‌افزاری با perf/VTune (بخش دوم)
۲۵٪	کیفیت روش علمی، رد فرضیه‌های جایگزین و بهبود برنامه (بخش سوم)
۱۵٪	عمق تحلیل و پیوند میان بخش‌ها
۱۵٪	کیفیت گزارش و تحویل
تا نیم نمره	بخش امتیازی: کنترل هسته‌ها و ریسمان‌ها